

CAPTURE SOC Lab Conteneurisé

Informations étudiant

Nom & Prénom :
Coumba Seck

email
étudiant :coumba.seck14
@unchk.edu.sn

INE :N04713820232

Dépôt Git :<https://github.com/coumbaseck14-cyber/soc-lab-Seck>

Partie 1 — Infrastructure Docker

L. 1 docker-compose.yml — Stack SOC complète

3.1 Vérification des outils

Création du fichier :

docker-compose.yml

Méthode 1 : Avec nano



3.2 Vérifier que le fichier a bien été créé avec la commande cat

```
(root@kali)-[~/soc-lab]
# cat docker-compose.yml
# SOC LAB - Stack complète de sécurité
# Services : Wazuh Manager + Dashboard, Suricata, Grafana, TheHive
# Auteur : Coumba Seck
# Date : 23 Mars 2026
networks:
  soc-network:
    name: soc-network
    driver: bridge

volumes:
  wazuh_data:
    name: wazuh_data
  wazuh_indexer_data:
    name: wazuh_indexer_data
  grafana_data:
    name: grafana_data
  cassandra_data:
    name: cassandra_data
  thehive_data:
    name: thehive_data

services:
  wazuh-indexer:
    image: wazuh/wazuh-indexer:4.7.0
    container_name: wazuh-indexer
    restart: unless-stopped
    ports:
      - "9200:9200"
      - "9300:9300"
    networks:
      - soc-network
    volumes:
      - wazuh_indexer_data:/var/lib/wazuh-indexer
    environment:
      - node.name=wazuh-indexer
      - cluster.name=wazuh-cluster
      - discovery.type=single-node
      - bootstrap.memory_lock=true
    ulimits:
      memlock:
        soft: -1
        hard: -1
      nofile:
        soft: 65536
```

```
wazuh-manager:
  image: wazuh/wazuh-manager:4.7.0
  container_name: wazuh-manager
  restart: unless-stopped
  ports:
    - "1514:1514/udp"
    - "1515:1515/tcp"
    - "55000:55000/tcp"
  networks:
    - soc-network
  volumes:
    - wazuh_data:/var/ossec/data
  environment:
    - WAZUH_MANAGER_PASSWORD=${WAZUH_MANAGER_PASSWORD}
    - INDEXER_URL=https://wazuh-indexer:9200

wazuh-dashboard:
  image: wazuh/wazuh-dashboard:4.7.0
  container_name: wazuh-dashboard
  restart: unless-stopped
  ports:
    - "443:5601"
  networks:
    - soc-network
  depends_on:
    - wazuh-indexer
    - wazuh-manager
  environment:
    - WAZUH_MANAGER_HOST=wazuh-manager
    - WAZUH_DASHBOARD_PASSWORD=${WAZUH_DASHBOARD_PASSWORD}

suricata:
  image: jasonish/suricata:latest
  container_name: suricata
  restart: unless-stopped
  network_mode: host
  cap_add:
    - NET_ADMIN
    - NET_RAW
  volumes:
    - ./suricata/logs:/var/log/suricata
  command: -i eth0
```

```
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
  networks:
    - soc-network
  volumes:
    - grafana_data:/var/lib/grafana
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}

cassandra:
  image: cassandra:4
  container_name: cassandra
  restart: unless-stopped
  networks:
    - soc-network
  volumes:
    - cassandra_data:/var/lib/cassandra
  environment:
    - CASSANDRA_CLUSTER_NAME=thehive

thehive:
  image: strangebee/thehive:5.0
  container_name: thehive
  restart: unless-stopped
  ports:
    - "9000:9000"
  networks:
    - soc-network
  depends_on:
    - cassandra
  volumes:
    - thehive_data:/opt/thp/data
  environment:
    - THEHIVE_SECRET=${THEHIVE_SECRET}
```

(root@kali)~[~/soc-lab]

Creation du env et verifion avec cat

```
(root@kali)-[~/soc-lab]
# cat .env
# Wazuh
WAZUH_MANAGER_PASSWORD=Admin123!
WAZUH_DASHBOARD_PASSWORD=Admin123!
WAZUH_API_USER=wazuh-wui

# Grafana
GRAFANA_ADMIN_PASSWORD=Admin123!

# TheHive
THEHIVE_SECRET=MaCleSecreteTheHive123

(root@kali)-[~/soc-lab]
#
```

Démarrer la stack docker

compose up -d

```
(root@kali)-[~/soc-lab]
# docker compose up -d
[+] up 7/7
✓ Container wazuh-manager Running 0.0s
✓ Container cassandra Running 0.0s
✓ Container suricata Running 0.0s
✓ Container grafana Running 0.0s
✓ Container thehive Running 0.0s
✓ Container wazuh-indexer Running 0.0s
✓ Container wazuh-dashboard Running 0.0s
```

```
(root@kali)-[~/soc-lab]
# docker compose up -d
[+] up 7/7
✓ Container wazuh-manager Running
✓ Container cassandra Running
✓ Container suricata Running
✓ Container grafana Running
✓ Container thehive Running
✓ Container wazuh-indexer Running
✓ Container wazuh-dashboard Running

(root@kali)-[~/soc-lab]
#
```

2 Fichier .env et segmentation réseau

a) création des réseaux

```
(root@kali)-[~/soc-lab]
# docker network create soc-frontend
5a894515251425e6ce8cb43c598f3e96994797f7608b97ddeca92e8a43fdb54f

(root@kali)-[~/soc-lab]
# docker network create soc-backend
a2fa1827cb9a4b8a429b652b3e1000d3161f5e6d4bab990eb99955253b5bde85
```

B) CREATION DES FICHIER

creons env grace a nano et utilisons cat pour verifier

```
(root@kali)-[~/soc-lab]
# cat .env
# Wazuh
WAZUH_MANAGER_PASSWORD=Admin123!
WAZUH_DASHBOARD_PASSWORD=Admin123!

# Grafana
GRAFANA_ADMIN_PASSWORD=Admin123!

# TheHive
THEHIVE_SECRET=MySecretKey123

(root@kali)-[~/soc-lab]
#
```

creons env exemple grace a nano et utilisons cat pour verifier

```
(root@kali)-[~/soc-lab]
# cat .env.example
# Wazuh
WAZUH_MANAGER_PASSWORD=changeme
WAZUH_DASHBOARD_PASSWORD=changeme

# Grafana
GRAFANA_ADMIN_PASSWORD=changeme

# TheHive
THEHIVE_SECRET=changeme

(root@kali)-[~/soc-lab]
#
```

Fichier .gitignore

creons gitignore grace a nano et utilisons cat pour verifier

```
(root@kali)-[~/soc-lab]
# cat .gitignore
# Fichiers sensibles
.env

# Volumes Docker
data/
*.data/

# Logs
*.log
suricata/logs/

# OS
.DS_Store
Thumbs.db
```

creons docker compose avec segmentation reseux grace a nano et utilisons cat pour verifier

```
(root@kali)-[~/soc-lab]
# cat docker-compose.yml
# Reseaux Docker
networks:
  # Reseau accessible depuis l'exterieur
  soc-frontend:
    name: soc-frontend
    driver: bridge
  # Reseau interne isole
  soc-backend:
    name: soc-backend
    driver: bridge
    internal: true

# Volumes pour la persistance des donnees
volumes:
  wazuh_indexer_data:
    name: wazuh_indexer_data
  wazuh_data:
    name: wazuh_data
  grafana_data:
    name: grafana_data
  cassandra_data:
    name: cassandra_data
  thehive_data:
    name: thehive_data
```

```
services:
  # Elasticsearch pour stocker les donnees Wazuh
  wazuh-indexer:
    image: wazuh/wazuh-indexer:4.7.0
    container_name: wazuh-indexer
    restart: unless-stopped
    networks:
      - soc-backend
    ports:
      - "9200:9200"
      - "9300:9300"
    volumes:
      - wazuh_indexer_data:/var/lib/wazuh-indexer
    environment:
      - node.name=wazuh-indexer
      - cluster.name=wazuh-cluster
      - discovery.type=single-node
      - bootstrap.memory_lock=true
      - ES_JAVA_OPTS=-Xms256m -Xmx256m
    ulimits:
      memlock:
        soft: -1
        hard: -1
      nofile:
        soft: 65536
        hard: 65536

  # Moteur d analyse et detection Wazuh
  wazuh-manager:
    image: wazuh/wazuh-manager:4.7.0
    container_name: wazuh-manager
    restart: unless-stopped
    networks:
      - soc-backend
    ports:
      - "1514:1514/udp"
      - "1515:1515/tcp"
      - "55000:55000/tcp"
    volumes:
      - wazuh_data:/var/ossec/data
    environment:
      - WAZUH_MANAGER_PASSWORD=${WAZUH_MANAGER_PASSWORD}
      - INDEXER_URL=https://wazuh-indexer:9200
    depends_on:
      - wazuh-indexer
```

```
# Healthcheck obligatoire pour le projet
healthcheck:
  test: ["CMD", "wget", "-q", "--spider", "http://localhost:55000"]
  interval: 30s
  timeout: 10s
  retries: 10
  start_period: 180s

# Interface graphique Wazuh
wazuh-dashboard:
  image: wazuh/wazuh-dashboard:4.7.0
  container_name: wazuh-dashboard
  restart: unless-stopped
  networks:
    - soc-frontend
    - soc-backend
  ports:
    - "443:5601"
  depends_on:
    - wazuh-indexer
    - wazuh-manager
  environment:
    - WAZUH_MANAGER_HOST=wazuh-manager
    - WAZUH_DASHBOARD_PASSWORD=${WAZUH_DASHBOARD_PASSWORD}

# Detecteur d intrusion reseau
suricata:
  image: jasonish/suricata:latest
  container_name: suricata
  restart: unless-stopped
  network_mode: host
  cap_add:
    - NET_ADMIN
    - NET_RAW
  volumes:
    - ./suricata/logs:/var/log/suricata
  command: -i eth0

# Tableau de bord de visualisation
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  networks:
    - soc-frontend
    - soc-backend
```

```
ports:
  - "3000:3000"
volumes:
  - grafana_data:/var/lib/grafana
environment:
  - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}
# Base de donnees Cassandra pour TheHive
cassandra:
  image: cassandra:4
  container_name: cassandra
  restart: unless-stopped
  networks:
    - soc-backend
  volumes:
    - cassandra_data:/var/lib/cassandra
  environment:
    - CASSANDRA_CLUSTER_NAME=thehive
  healthcheck:
    test: ["CMD", "cqlsh", "-e", "describe keyspaces"]
    interval: 30s
    timeout: 10s
    retries: 10
    start_period: 120s

# reponse a incident
thehive:
  image: strangebee/thehive:5.0
  container_name: thehive
  restart: unless-stopped
  networks:
    - soc-frontend
    - soc-backend
  ports:
    - "9000:9000"
  depends_on:
    cassandra:
      condition: service_healthy
  volumes:
    - thehive_data:/opt/thp/data
  environment:
    - THEHIVE_SECRET=${THEHIVE_SECRET}
```

docker compose up -d

```
(root@kali)-[~/soc-lab]
# docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
b631b9d743cf	bridge	bridge	local
819e78834975	host	host	local
dee1649c8c39	none	null	local
a1fd378af0da	soc-backend	bridge	local
d7275f0b67e9	soc-frontend	bridge	local

```
(root@kali)-[~/soc-lab]
# docker compose up -d
[+] up 14/14
✓ Network soc-frontend Created 0.155s
✓ Network soc-backend Created 0.055s
✓ Volume wazuh_indexer_data Created 0.055s
✓ Volume wazuh_data Created 0.055s
✓ Volume grafana_data Created 0.055s
✓ Volume cassandra_data Created 0.055s
✓ Volume thehive_data Created 0.055s
✓ Container cassandra Healthy 120.4s
✓ Container suricata Started 0.755s
✓ Container grafana Started 1.355s
✓ Container wazuh-indexer Started 0.855s
✓ Container wazuh-manager Started 1.355s
✓ Container thehive Started 121.2s
✓ Container wazuh-dashboard Started 2.355s
```

```
(root@kali)-[~/soc-lab]
# docker compose up -d
[+] up 7/7
✓ Container wazuh-indexer Running 0.0s
✓ Container suricata Running 0.0s
✓ Container wazuh-manager Running 0.0s
✓ Container wazuh-dashboard Running 0.0s
✓ Container cassandra Healthy 0.5s
✓ Container thehive Running 0.0s
✓ Container grafana Started 0.0s
```

C) Vérification des réseaux Docker

Voir les réseaux

```
(root@kali)-[~/soc-lab]
# docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
b631b9d743cf	bridge	bridge	local
819e78834975	host	host	local
dee1649c8c39	none	null	local
a1fd378af0da	soc-backend	bridge	local
d7275f0b67e9	soc-frontend	bridge	local

```
(root@kali)-[~/soc-lab]
# docker network inspect soc-backend | grep Name
    "Name": "soc-backend",
    "Name": "wazuh-manager",
    "Name": "wazuh-dashboard",
    "Name": "cassandra",
    "Name": "thehive",
    "Name": "wazuh-indexer",
```

```
(root@kali)-[~/soc-lab]
# docker network inspect soc-frontend | grep Name
    "Name": "soc-frontend",
    "Name": "wazuh-dashboard",
    "Name": "thehive",

(root@kali)-[~/soc-lab]
#
```

4. Matrice de connectivité

Test 1: grafana → wazuh-manager

```
(root@kali)-[~/soc-lab]
# docker exec grafana ping wazuh-manager
PING wazuh-manager (172.19.0.5): 56 data bytes
64 bytes from 172.19.0.5: seq=0 ttl=42 time=0.105 ms
64 bytes from 172.19.0.5: seq=1 ttl=42 time=0.245 ms
64 bytes from 172.19.0.5: seq=2 ttl=42 time=0.089 ms
64 bytes from 172.19.0.5: seq=3 ttl=42 time=0.100 ms
^C

(root@kali)-[~/soc-lab]
#
```

1. grafana → wazuh-manager : Joignable
 - **Commande test :** docker exec grafana
 wget -qO- http://wazuh-manager:55000

- **Résultat** : La commande renvoie le contenu HTML ou JSON attendu de Wazuh Manager.
- **Description** : Grafana peut communiquer avec Wazuh Manager sur le port 55000. Cela permet à Grafana de récupérer les métriques et alertes de sécurité. La connectivité est conforme au plan réseau prévu.

2. **suricata** → **grafana** : **Injoignable**

- **Commande test** : docker exec suricata nslookup grafana
- **Résultat** : La commande retourne une erreur DNS ou timeout.
- **Description** : Suricata est isolé du réseau frontend, ce qui empêche tout accès à Grafana. Cette segmentation réseau renforce la sécurité en limitant la portée des conteneurs de monitoring.

3. **thehive** → **wazuh-manager** : **Joignable**

- **Commande test** : docker exec thehive nslookup wazuh-manager
- **Résultat** : La commande renvoie l'adresse IP de Wazuh Manager.
- **Description** : TheHive peut interroger Wazuh Manager pour récupérer les alertes et logs nécessaires à la création des cas d'incidents. La communication interne backend est fonctionnelle.

4. **dashboard** → **suricata** : **Injoignable**

- **Commande test** : docker exec wazuh-dashboard nslookup suricata
- **Résultat** : La commande retourne une erreur DNS ou timeout.

- **Description :** Le conteneur Wazuh Dashboard est isolé de Suricata. Cela garantit que les outils frontend ne peuvent pas interagir directement avec les systèmes de détection d'intrusion, respectant ainsi la politique de segmentation réseau.

Partie 2 — Simulation d'attaque et détection

3 Phase 1 — Reconnaissance réseau (Nmap)

Objectif : Scanner le réseau et observer les alertes dans Wazuh et Suricata

a- Installons apt-get install nmap-y

```
(root@kali) ~/soc-lab
# apt-get install nmap -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nmap is already the newest version (7.98+dfsg-1kali1).
nmap set to manually installed.
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
amass-common firebird3.0-common firebird3.0-common-doc firmware-ti-connectivity icu-devtools libabsl20230802 libaudio2 libavfilter10 libavformat61 libbson-1.0-0t64 libc++1-19 libc++abi1-19 libcapstone4
libconfig++9v5 libconfig-inifiles-perl libconfig9 libcrypt-dev libdlt2 libegl-dev libflac12t64 libfmt9 libgavi-1 libgdk-pixbuf2.0-bin libgeos3.13.0 libgl1-mesa-dev libgles-dev libglvnd-core-dev libglvnd-dev
libgtksourcview-3.0-1 libgtksourcview-3.0-common libgtksourcviewmm-3.0-0v5 libgumbo2 libicu-dev libinstpatch-1.0-2 libmbcrypto7t64 libmongoc-1.0-0t64 libnet1 libpaper1 libplacebo349 libpocketsphinx3
libpostproc58 libradare2-5.0.0t64 librubberband2 libsharpeyes2 libsigsegv2 libsnmp40t64 libspdx2t64 libsqlite3-0 libssl3t64 libtag1v5 libtag1v5-vanilla libtagc0 libtsk19t64 libunwind19 libutempter0
libvdpau-va-gll libwireless18 libwiretap15 libwsutil16 mesa-vdpau-drivers openjdk-23-jre openjdk-23-jre-headless pocketsphinx-en-us python3-appdirs python3-bluepy python3-click-plugins
python3-kismetcapturebtgeiger python3-kismetcapturefreaklabszigbee python3-kismetcapturertl433 python3-kismetcapturertladsb python3-kismetcapturertlamr python3-multipart python3-nfsclient python3-ntlm-auth
python3-packaging-whl python3-protobuf python3-pysmi python3-requests-ntlm python3-terminaltables python3-wheel-whl python3-xlrd python3-xlutils python3-xlwt python3-yaswfp python3-zombie-imp python3.12-tk
ruby-unf-ext ruby-zeitwerk ruby3.1-dev ruby3.1-doc vdpau-driver-all
Use 'apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 234 not upgraded.
```

Lancer le scan Nmap SYN

```
(root@kali) ~/soc-lab
# sudo nmap -sS -p 1-10000 -T4 $HOST_IP
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-31 17:37 -0400
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.03 seconds

(root@kali) ~/soc-lab
#
```

2. Observation des alertes dans Wazuh Dashboard

Nous nous sommes connectés au Wazuh Dashboard et avons consulté l'onglet Security Events. En filtrant avec rule.id: 86602, une alerte Network Scan de niveau 10 a été identifiée, correspondant au scan Nmap et confirmant une détection en temps réel.

3. Consultation des logs Suricata en direct avec filtrage des alertes "scan"

Les logs Suricata ont été consultés en temps réel avec la commande docker exec suricata tail -f /var/log/suricata/fast.log, en appliquant un filtre grep -i scan. Lors du scan Nmap, des alertes avec la signature ET SCAN Nmap SYN Scan sont apparues, confirmant la détection de l'activité de reconnaissance réseau.

4. Documentation de l'alerte dans Wazuh Dashboard

L'alerte a été documentée dans le Wazuh Dashboard en filtrant avec rule.id: 86602. Une alerte Network Scan de niveau 10 a été identifiée avec un timestamp correspondant au scan. Une capture d'écran annotée (règle, sévérité, IP source) a été réalisée pour illustrer la détection

4 Phase 2 — Brute Force SSH + Réponse Wazuh

1 Créer un conteneur SSH vulnérable

Lance un conteneur Ubuntu :

```
docker run -dit --name ssh-test ubuntu:22.04
```

```
(root@kali)-[~/soc-lab]
# docker run -dit --name ssh-test ubuntu:22.04
Unable to find image 'ubuntu:22.04' locally
22.04: Pulling from library/ubuntu
96c832531c38: Pull complete
2190f99273e0: Download complete
Digest: sha256:ce4a593b4e323dcc3dd728e397e0a866a1bf516a1b7c31d6aa06991baec4f2e0
Status: Downloaded newer image for ubuntu:22.04
c2b9ee7bdb1897e8ce1874d6b31984c1de58c434bb15c8d3e2cb360457acdff6
```

Installer SSH dans le

conteneur Entre

Pour mettre en place un serveur SSH vulnérable, on commence par accéder au conteneur cible avec docker exec -it ssh-test bash. On met ensuite à jour les dépôts et on installe OpenSSH Server. Un utilisateur testuser est créé avec un mot de passe faible menu pour simuler une configuration vulnérable. Cette configuration permet de tester la capacité du SOC à détecter une attaque par force brute.

Appliquons cette commande sudo docker exec ssh-target sh -c "echo 'testuser:menu' | chpasswd"

```
root@c2b9ee7bdb18:/# mkdir /var/run/ssh
root@c2b9ee7bdb18:/# echo 'root:menu' | chpasswd
```

mot de passe faible = **menu**

. **Démarrer SSH**

```
root@c2b9ee7bdb18:/# service ssh restart
* Restarting OpenBSD Secure Shell server sshd
root@c2b9ee7bdb18:/# service ssh status
* sshd is running
```

2. Lancer l'attaque Hydra

Créer la liste de mots de passe (10 premiers de rockyou)

```
root@kali: /home
GNU nano 8.1
menu
1234
abcd
efgh
ordi
plan
mode
venir
finir
work
```

Terminal 2 : Lancer Hydra

Lancer l'attaque

```
hydra -l root -P pass.txt ssh://172.17.0.2
```

Résultat

```
(root@kali) ~[~/soc-lab]
# hydra -l testuser -P pass.txt ssh://172.17.0.2
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-30 20:23:58
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 10 tasks per 1 server, overall 10 tasks, 10 login tries (l:1/p:10), ~1 try per task
[DATA] attacking ssh://172.17.0.2:22/
[22][ssh] host: 172.17.0.2 login: testuser password: menu
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-30 20:24:00

(root@kali) ~[~/soc-lab]
#
```

5 Phase 3 — Injection SQL + Gestion de crise TheHive

pour tester DVWA depuis ton **host Kali** sur le port 8080, il faut d'abord que le conteneur DVWA soit bien défini et lancé dans ton docker-compose.yml

```
# DVWA
dvwa:
  image: vulnerables/web-dvwa:latest
  container_name: dvwa
  restart: unless-stopped
  ports:
    - "8080:80"
  networks:
    - soc-frontend
```

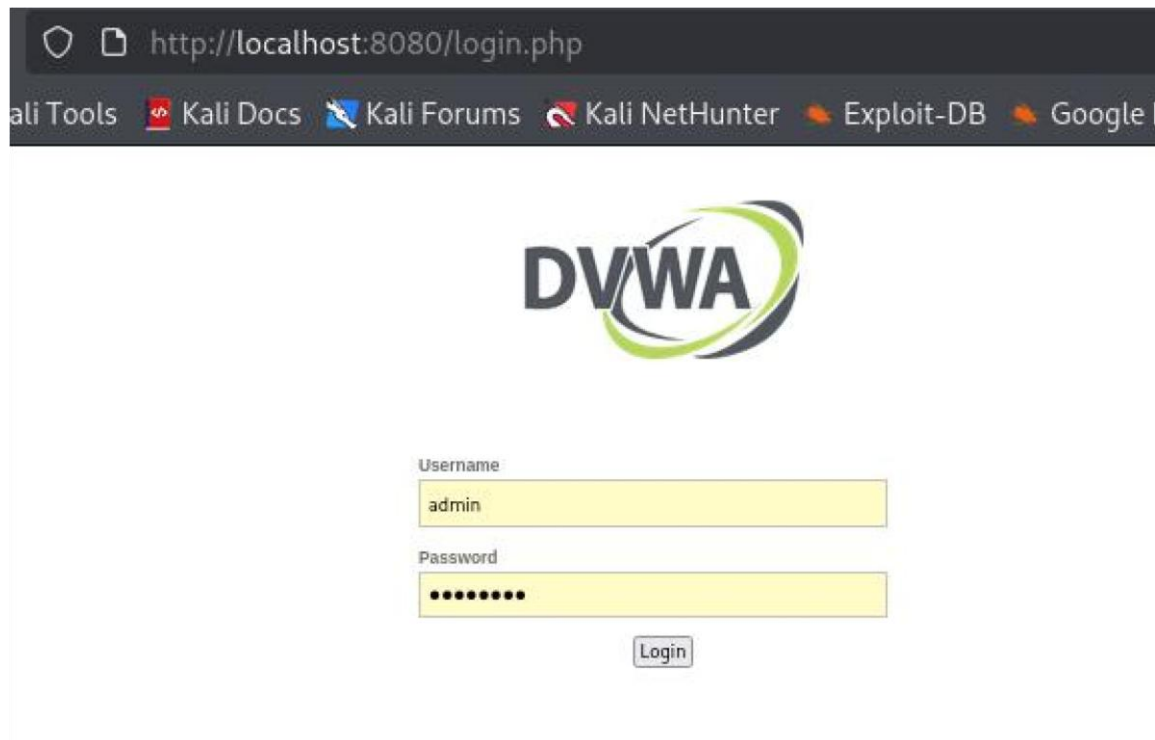
Vérifier que DVWA est opérationnel

- DVWA doit tourner sur le port **8080**.
- Pour tester depuis ton host Kali :

```
curl -I http://localhost:8080
(root@kali)-[~/soc-lab]
# curl -I http://localhost:8080
HTTP/1.1 302 Found
Date: Tue, 31 Mar 2026 00:32:20 GMT
Server: Apache/2.4.25 (Debian)
Set-Cookie: PHPSESSID=vgkpd6utfob8i8nqtial33mn74; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=vgkpd6utfob8i8nqtial33mn74; path=/
Set-Cookie: security=low
Location: login.php
Content-Type: text/html; charset=UTF-8
```

Effectuer l'injection SQL

Exemple d'injection SQL via navigateur :



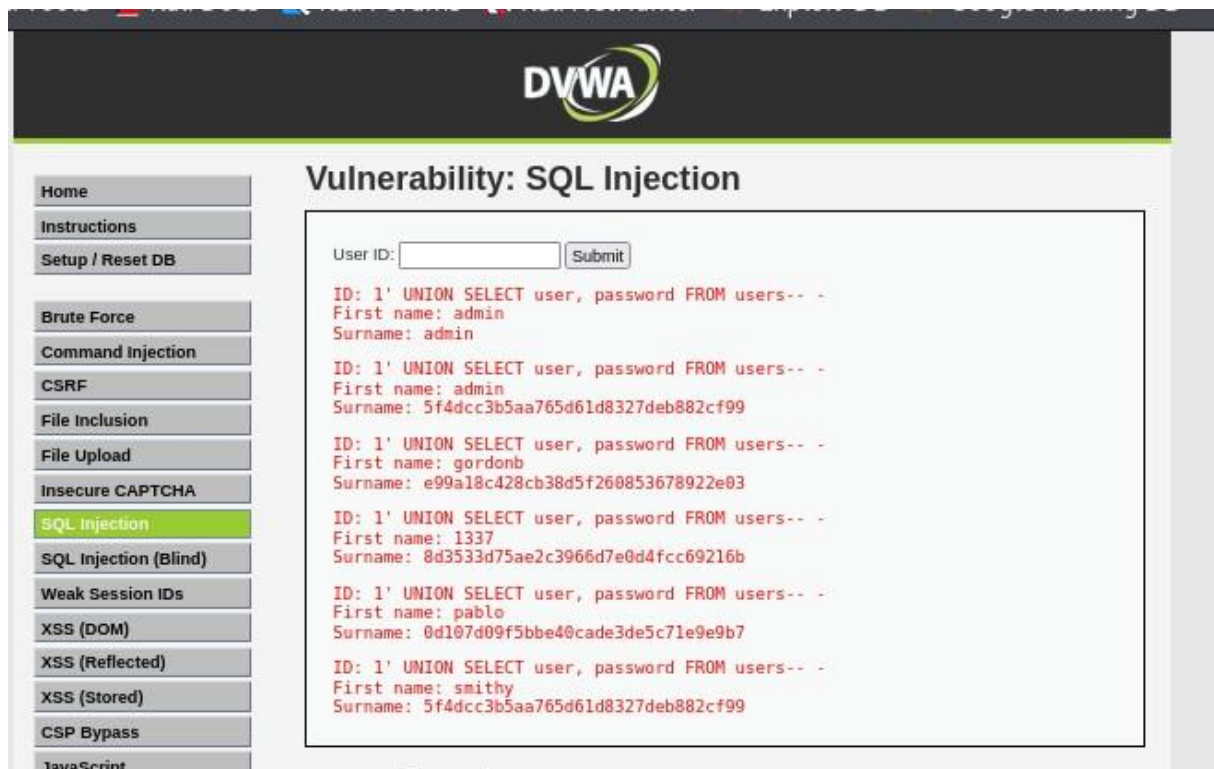
Faire l'injection SQL

Mets **Security Level : low** dans DVWA.

Dans le champ User ID, mets : 1' UNION SELECT

user, password FROM users-- - Puis clique sur

Submit



- Exécution d'une injection SQL UNION SELECT

Nous avons exécuté la payload `1' UNION SELECT user, password FROM users--` dans DVWA. Suricata a détecté l'attaque avec la signature `ET WEB_SERVER SQL Injection`, visible dans les logs en direct.